Nuvio Dublaj: Yeni Site Ekleme Kılavuzu

Oct 24, 2026 · @Ali

Genel bakış

Aynı dosyaya istediğin kadar site ekleyebilirsin: her site `providers` listesine eklenen ayrı bir kayıttır. Uygulama bunları **yukarıdan aşağıya sırayla** dener ve **ilk ses bulan kazanır**. Yani üç site eklemek "hepsinden ses çekip hepsini menüde göstermek" değil, "biri bulamazsa sıradakine bak" demektir. Üçünü birden menüde göstermek uygulama kodunda değişiklik ister.

Her sağlayıcı için akış aynı:

1. Uygulama içeriğin IMDb kimliğini, başlığını, yılını, sezon/bölümünü ve süresini alır.
2. Sağlayıcının `queries` listesiyle sitede arar (film için `movie`, dizi için `series`).
3. Arama sonucu sayfasından en fazla 3 aday bağlantı çıkarır (`search.pick`).
4. Her adayda `steps` listesini çalıştırır; filmlerde sayfadaki süre `check` ile doğrulanır.
5. Çıkan link `hls_audio` ile kontrol edilir. Yalnızca ses ise videoya eklenir, video içeriyorsa reddedilir.

`movie` ya da `series` bloğunu boş bırakırsan o sağlayıcı o içerik türünde sonuç vermez. Dizibox gibi sadece dizi sitesi için yalnızca `series`, film sitesi için yalnızca `movie` yazman yeterli.

Yeni site ekleme: adım adım

Bir site eklemek, o sitede ses linkine giden yolu önce tarayıcıda bulup sonra JSON'a adım adım yazmak demektir. Yol her sitede farklıdır; aşağıdaki sıra hepsinde aynı.

1. **Hazırlık.** Bilgisayarında Python 3, depodaki `dubtool.py` ve tarayıcının geliştirici araçları (F12) yeterli.
2. **Arama adresini bul.** Sitenin arama kutusuna bir film yaz, adres çubuğuna bak. `https://ornek.com/arama/inception` ise `search.url` = `{base}/arama/{q}`. Adres `?s=inception` biçimindeyse `{base}/?s={q}` yaz.
3. **Sonuç bağlantısını yakalayan regex'i yaz (`search.pick`).** Arama sonuç sayfasının kaynağını `arama_sonucu.html` olarak kaydet. İlk sonucun `href` değerini yakalayan, grup 1'i bağlantı olan bir regex yaz. Denemek için: `python3 dubtool.py --provider siteadi --test-pick arama_sonucu.html`

4. **Ses linkine giden zinciri bul.** Bir film sayfasında oynatmayı başlat, F12 → Network sekmesinde `m3u8` ya da `mp3` ile filtrele. Linkin hangi isteğin cevabında çıktığını bul. Zincir genelde film sayfası → player adresi → player sayfası → ses linki şeklindedir.
5. **Zinciri `steps` olarak yaz.** Her halka bir adımdır: sayfayı indir (`http_get`), içinden parça çek (`regex` , `json_path`), gerekiyorsa çöz (`base64` , `rot13` , `reverse` , `shift_key`). Adımların tam listesi sonraki bölümlerde.
6. **Sağlayıcıyı ekle.** `providers` dizisine yeni bir nesne koy, `id` ve `label` ver (sonraki bölümde şablon var). Etiket, menüde hangi siteden geldiğini göstermek için kullanılır.
7. **Dene ve yayınla.** Önce `dubtool.py` ile bilgisayarda, sonra GitHub'a yükleyip telefonda dene.

Sıra önemlidir: en güvenilir aldığın siteyi listenin en üstüne koy, çünkü ilk bulan kazanır.

Sağlayıcı alanları

Her sağlayıcı kaydı aynı alanlara sahiptir; sitelere göre yalnızca değerler ve `steps` değişir. Dosyanın en üstünde ayrıca `schemaVersion` (1) ve `autoSearch` (`false` yaparsan videoda otomatik arama kapanır, menüdeki düğme çalışmaya devam eder) bulunur.

Alan	Anlamı	Örnek
`id`	Sağlayıcının benzersiz adı. `dubtool.py --provider` bununla çağrılır	`hdfilmcehennemi`
`enabled`	`false` yaparsan sağlayıcı silinmeden geçici kapanır	`true`
`label`	Ses parçasına verilen ad. Site adını buraya yaz	`Türkçe Dublaj · HDFC`
`lang`	Aranan dil	`tr`
`base`	Sitenin ana adresi, `https` ile başlamalı	`https://ornek.com`
`audioHeaders`	Ses akışı yüklenirken gönderilen başlıklar. `{base}` kullanılabilir	`Origin` , `Referer`
`movie.queries`	Film aramasında denenen kelimeler, sırayla	`["{imdb}", "{title}"]`
`series.queries`	Dizi aramasında denenen kelimeler	`["{title} {season}. Sezon {episode}. Bölüm"]`

Alan	Anlamı	Örnek
`search.url`	Arama adresi	`{base}/arama/{q}`
`search.maxCandidates`	Sonuçtan kaç aday denenecek	`3`
`search.pick`	Sonuç sayfasından bağlantı çeken regex listesi, grup 1 = bağlantı	bkz. şablon
`steps`	Ses linkine giden adımlar	sonraki bölüm
`result`	Son ses linkinin değişken adı (`as` ile verdiğin isim)	`audioFinal`

Sorgularda ve adreslerde kullanılabilen değişkenler: `{base} {imdb} {title} {title_raw} {year} {season} {episode} {durationMin} {q} {page}` ve `as` ile adlandırdığın her şey. Boş bir değişken içeren sorgu atlanır; örneğin filmde `{season}` boştur. `{imdb}` ile aramayı yalnızca site IMDb kimliğini arama kutusunda destekliyorsa koy.

Adım (op) referansı

`steps` içinde yalnızca aşağıdaki işlemler kullanılabilir; tarif kod çalıştırmaz, yalnızca bu hazır işlemleri sıralar.

op	Parametreler	Ne yapar
`http_get`	`url` , `referer` , `as`	Sayfayı indirir
`regex`	`pattern` ya da `patterns` (liste, sırayla denenir), `group` , `dotAll` , `ignoreCase`	Metinden parça çeker
`json_path`	`path` (örn. `a.b[0].c`)	JSON'dan değer alır
`rot13` , `reverse` , `trim`	yok	Harf kaydırma, ters çevirme, boşluk silme
`base64`	`charset` : `latin1` (varsayılan) ya da `utf-8`	Çözer
`shift_key`	`key` , `mod` , `add` , `dir`	Her karakteri anahtara göre kaydırır

op	Parametreler	Ne yapar
`replace`	`pattern`, `with`, `shift`	Desene uyan kısmı değiştirir. `with` içinde `\\1`, `\\2` grup başvurusu kullanılır
`require`	`pattern`	Sonuç desene uymuyorsa adım durur. Yanlış dili elemek için
`check`	`from`, `pattern`, `expect`, `tolerance`	Sayfadaki sayıyı (örn. süre) bilinenle karşılaştırır
`set`	`value` (`{cur}` önceki sonuç)	Şablonla yeni metin üretir
`hls_audio`	`lang`, `name`	Link video içeren master liste ise ayrı ses parçasını seçer; ses parçası yoksa durur

Kurallar:

- Her adım girdiyi `from` ile alır. Yazmazsan önceki adımın sonucu kullanılır.
- `as` ile sonuca isim verirsin, sonra `{isim}` ya da `from` ile kullanırsın.
- Regex'leri JSON içinde çift ters bölüyle yaz: `\\d`, `\\s`.
- En fazla 40 adım; yalnızca `https` adresleri.
- Zincirin sonunda `require` ile dili, `hls_audio` ile ses olduğunu doğrula. Bu iki adım yanlış dil ya da video akışının eklenmesini engeller.
- Sitenin kullandığı şifreleme bu listede yoksa (örneğin AES) uygulamaya yeni bir `op` eklemek ve APK'yı yenilemek gerekir.

Boş sağlayıcı şablonu

Aşağıdaki kayıt yalnızca biçim örneğidir; `ornek.com` gerçek bir site değildir ve `steps` içindeki adımları senin sitenin yoluna göre yeniden yazman gerekir. Kaydı `providers` dizisinde mevcut sağlayıcının kapanan `}` işaretinden sonra, araya virgül koyarak ekle.

```
{
  "id": "ornek",
  "enabled": true,
  "label": "Türkçe Dublaj · Örnek",
  "lang": "tr",
  "base": "https://ornek.com",
  "audioHeaders": { "Origin": "{base}", "Referer": "{base}/" },
  "movie":  { "queries": ["{imdb}", "{title}"] },
  "series": { "queries": ["{title} {season}. Sezon {episode}. Bölüm"] },
```

```
  "search": {
    "url": "{base}/arama/{q}",
    "maxCandidates": 3,
    "pick": [ "<div class=\"item\"><a href=\"([^\"]+)\"" ]
  },
  "steps": [
    { "op": "http_get", "url": "{page}", "referer": "{base}/", "as": "html"
},
    { "op": "check", "from": "html", "pattern":
"\"duration\":\\s*\"PT(\\d+)M\"", "expect": "durationMin", "tolerance": 5 },
    { "op": "regex", "from": "html", "pattern": "data-player=\"([^\"]+)\"",
"group": 1 },
    { "op": "base64", "as": "playerUrl" },
    { "op": "http_get", "url": "{playerUrl}", "referer": "{base}/" },
    { "op": "regex", "pattern": "\"audio\"\\s*:\\s*\"([^\"]+)\"", "group": 1,
"as": "audioUrl" },
    { "op": "require", "from": "audioUrl", "pattern": "tra\\d+$" },
    { "op": "hls_audio", "from": "audioUrl", "lang": "tr", "as": "audioFinal"
}
  ],
  "result": "audioFinal"
}
```

Uyarlarken dikkat edilecekler:

- `require` deseni (`tra\\d+$`) fullhdfilmizlesene'nin link biçimine göre yazıldı. Yeni sitede Türkçe sesin linkinin nasıl bittiğini kendi örneklerinle bulup o desene göre değiştir; uymayan bir desen her sonucu reddeder. Siteyi tanıyan bir desen yoksa `require` adımını koyma, `hls_audio` yine videoyu eler.
- Sadece dizi sitesi eklerken `movie` bloğunu, sadece film sitesi eklerken `series` bloğunu sil.
- Dizilerde süre bilinmediği için `check` adımı otomatik atlanır.

Site analizi: ne toplanır, nasıl denenir

Yeni bir sitede en zor kısım ses linkinin yolunu bulmaktır; bunun için taratıcıdan dört şey toplarsın ve `dubtool.py` ile her adımı bilgisayarda denersin.

Tarayıcıda bulunacaklar

1. **Arama sonucu sayfasının kaynağı.** Sağ tık → Sayfa kaynağını göster. `search.pick` regex'i bundan yazılır.

2. **Film sayfasının kaynağı.** İçinde `m3u8`, `iframe`, `atob(`, `file:`, `eval(` kelimelerini ara. Player adresi ya da ses linki düz mi, şifreli mi (rastgele harf yığını) buradan anlaşılır. Sayfada `"duration"` gibi bir süre alanı varsa `check` adımı bunu kullanır.
3. **Player sayfasının kaynağı.** Film sayfasındaki iframe adresini ayrı sekmede açıp kaynağına bak. Player çoğu zaman Referer ister: önce film sayfasından oynat, sonra player adresini aç.
4. **Ses isteğinin başlıkları.** Network sekmesinde ses linkine sağ tık → Copy as cURL. `referer` ve `origin` satırlarını `audioHeaders` içine yaz.

Bilgisayarda denemek

```
# film: aramadan başlar
python3 dubtool.py --provider siteadi --imdb tt1375666 --title "Inception" --
duration 148

# dizi
python3 dubtool.py --provider siteadi --series --title "Dizi Adı" --season 1
--episode 2

# aramayı atla, tek bir sayfadan başla (adımları ayıklamak için en
kullanışlısı)
python3 dubtool.py --provider siteadi --page
https://ornek.com/film/inception/ --duration 148

# search.pick regex'ini kayıtlı HTML'de dene
python3 dubtool.py --provider siteadi --test-pick arama_sonucu.html

# her adımın tam çıktısını göster
python3 dubtool.py --provider siteadi --page
https://ornek.com/film/inception/ --dump

# bulunan linkin ses mi video mu olduğunu söyler
python3 dubtool.py --provider siteadi --inspect "https://...ses-linki" --
referer https://ornek.com/
```

Araç her adımın çıktısını gösterir ve durduğu adımı yazar (örn. `[3] regex eşleşmedi ... -> DURDU`). O adımı düzeltip yeniden çalıştır. Sonunda bir `https://` linki çıkarsa `--inspect` ile ses olduğunu kontrol et ve tarayıcıda ya da bir oynatıcıda açıp dublajın duyulduğunu doğrula.

Takılırsan bana gönderilecekler

Sitenin adımı çözebilmem için yukarıdaki dört kaynağı `.txt` olarak yükle. Ses linkinin düz mı şifreli mi olduğunu ve bir örnek ses linkini de yaz. Bu sitelere buradan erişemiyorum; adımları senin gönderdiğin kaynaklardan çıkarır ve `dubtool.py` çıktılarıyla doğrularım.

hdfilmcehennemi ve dizibox için durum

Her üç siteyi aynı dosyaya koyabilirsin, ama hdfilmcehennemi ve dizibox için hazır tarif yazamadım: bu sitelere bu ortamdan erişemiyorum ve sayfalarını hiç görmedim. Tahminle yazılmış bir tarif ya hiç sonuç vermez ya da yanlış filmi getirir. Sayfa kaynaklarını gönderirsen tarifi onlardan çıkarırım.

Önerilen yerleşim

Sıra	Site	Blok	Neden
1	fullhdfilmizlesene	`movie` + `series`	Çalışan, test edilmiş tarif
2	hdfilmcehennemi	`movie` (dizisi varsa `series` de)	Film yedeği
3	dizibox	yalnızca `series`	Dizi sitesi, filmde boş kalır

Bir sitenin tarife uyup uymadığını belirleyen şey

Tarif, Türkçe sesin videodan ayrı bir ses akışı olarak sunulduğu sitelerde çalışır. fullhdfilmizlesene bunu yapıyor: ses linki ayrı ve sonu `tra1` ile bitiyor. Birçok sitede ise dublaj video akışının içine gömülüdür. O zaman `hls_audio` adımı linki video diye reddeder ve tarif sonuç bulamaz. Uygulamada sesi video akışından ayırma özelliği yok; bunu eklemek ayrı bir uygulama güncellemesi olur.

Tarifin aşamayacağı engeller

- Cloudflare gibi bot koruması olan siteler: uygulama bir tarayıcı değil, sıradan HTTP istekleri atar.
- Linki JavaScript çalışınca üreten sayfalar (paketlenmiş `eval(...)` kodu gibi): tarif JavaScript çalıştırmaz.
- Listedeki işlemlerde olmayan şifreleme (AES gibi): yeni bir `op` ve yeni APK gerekir.

Bu engellerden biri çıkarsa `dubtool.py` hangi adımda durduğunu gösterir; o adımın çıktısını gönderirsen çözülebilir mi, uygulamaya yeni işlem mi gerekir söyleyebilirim.

Test, yayınlama ve sorun giderme

Yeni siteyi önce bilgisayarda, sonra telefonda dene; JSON'da tek bir virgül hatası tüm dosyayı geçersiz kılar ve uygulama eski kopyayı kullanmaya devam eder, yani eski sağlayıcılar çalışmaya devam eder ama yeni eklediğin hiçbir şey uygulanmaz.

Yayınlamadan önce

1. JSON'u doğrula: `python3 -c "import json;json.load(open('dublaj_kaynaklari.json',encoding='utf-8'))"`

2. Eski sağlayıcının bozulmadığını kontrol et: `python3 test_recipe.py`
3. Yeni sağlayıcıyı `dubtool.py` ile en az bir film ve bir dizide dene (varsa).

Yayınlama

1. Dosyayı `gitlihub0/nuvio-dublaj` deposuna yükle (commit et).
2. GitHub `raw` içeriğini yaklaşık 5 dakika önbelleğe alır; 5 dakika bekle.
3. Uygulamada bir video aç → ses menüsü → **Türkçe dublaj ara**. Bu düğme her seferinde önce listeyi indirir. Otomatik aramada ise telefondaki kopya 6 saatten eskiyse yenilenir.
4. Menüde parçanın yanında `label` değerinin görünüp görünmediğine bak. Görünmezse etiketin menüye aktarılması için küçük bir uygulama değişikliği gerekir.

Log ile hata ayıklama

```
adb logcat | grep -E
"DubSource|DubPipeline|DubSearch|ExternalAudio|DubConfig"
```

Log	Anlamı
`Config parse failed` / `Downloaded config is invalid`	JSON hatalı, eski kopya kullanılıyor
`[id] search failed`	Arama isteği hata verdi (adres, ağ ya da site engellemesi)
`[id] trying <url>`	Aday bulundu, adımlar çalışıyor
`step 'regex' stopped: no match`	O adımın regex'i sayfaya uymuyor, site değişmiş olabilir
`step 'check' stopped`	Süre uyuşmadı: yanlış film ya da dar tolerans
`step 'require' stopped`	Link beklenen dilde bitmiyor
`no separate audio rendition`	Link video içeriyor, ayrı ses parçası yok
`[id] audio found`	Ses linki bulundu

Sık görülen durumlar

- Hiçbir şey bulunmuyor: `--page <film sayfası>` ile adımları tek tek dene.
- Arama sonucu çıkmıyor: kayıtlı HTML'de `--test-pick` ile `search.pick` regex'ini dene.
- Doğru film reddediliyor: `check` adımının `tolerance` değerini artır ya da `pattern`'ı sitenin süre biçimine göre düzelt.
- Yanlış film geliyor: `tolerance` değerini düşür, `{title}` sorgusunu kaldır, yalnızca `{imdb}` kullan.

- Link bulunuyor ama ses gelmiyor: `audioHeaders` (Referer, Origin) eksik ya da link süresi dolmuş olabilir.